M2104 Object-Oriented Design & Modelling – Course Text – Explaining a Program
Session 2

1 Lesson : Class diagram review

1.1 Review

Objects represent entities (individuals) in the program's environment. They contain operations (have a behaviour, provide services) and also the data necessary for the execution of these operations; these are the **attributes** (fields) and **methods** of the objects. Objects are defined by **classes** that define the characteristics of a kind of objects. Classes can also define operations and data that are not associated to any object but to the type itself, the **class methods and attributes**. An object-oriented program uses the relationships between classes such as objects that refer to other objets : **associations**, as well as methods that call methods or use objects of other classes : **use links**. Attributes and methods have a **visibility** that allow to distinguish things provided to other classes (public) or specific to the class (private), among others.

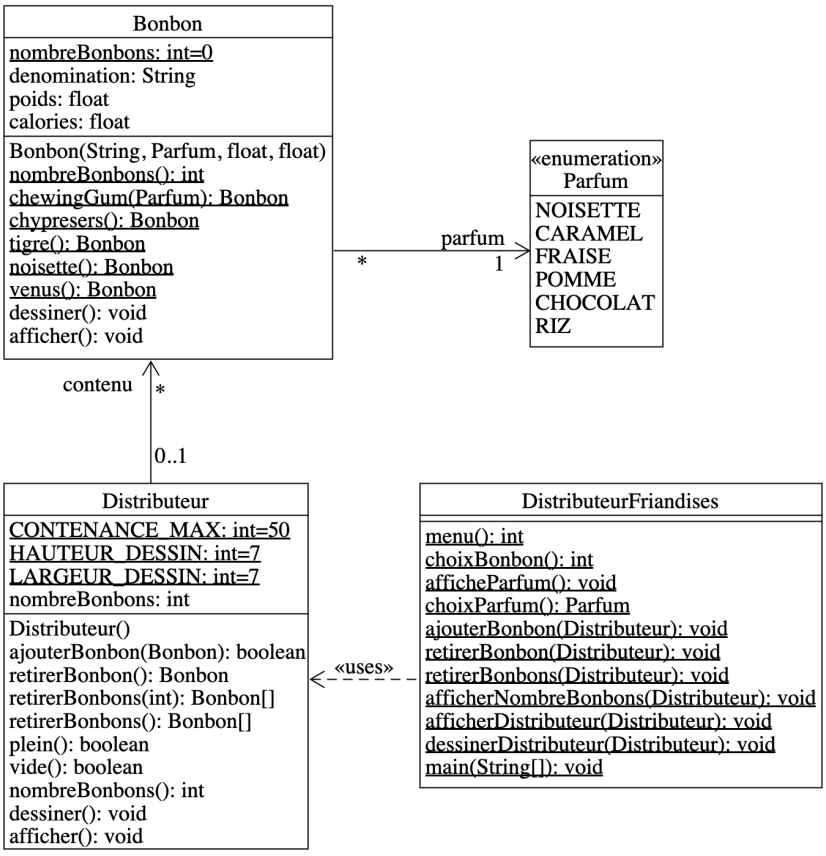
The class diagram represent classes as **rectangle boxes** containing up to three zones : the name of the class (mandatory), the attributes, and the methods (as well as the constructors). Visibilities are noted as **symbols** such as + and −. Associations are links (continuous lines) decorated by names (optionals and often replaced by roles), **roles** (at least one) and **multiplicities** (mandatory); roles are the names of the references to objects in the code, and multiplicities make it easy to know how many objects can participate in the association, given one object of the other class. Multiplicities and roles are **associated to the nearest class**. Use links are represented by a dashed line and the stereotype *uses*.

Plural multiplicities denote most of the time the use of *containers* in Java code : arrays, lists, sets, or associative mappings. An **association class** allow to characterise using an objects data relating to one pair of associated objects, it is represented by a class linked with a dashed line to a plural association in the class diagram, and can be implemented using an associative mapping.

1.2 Addition : navigability

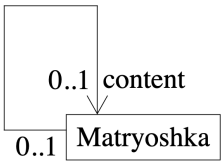
An association or use link with **one-way navigability** exists whenever an object of some class *A* refers to (or uses) a class *B*, but the reverse is not true. **navigability** is marked on the class diagram by an **arrow** on the link involved. A single arrow denotes **one-way navigability**, one at each end of the link, **two-way navigability**, and no arrow a non-specific navigability (this information is not always present on class diagrams). In the case of one-way navigability, in Java, one of the classes has a reference to the other, but not the other way around.

This is the class diagram of the practical *candy vending machine*, updated with the navigability of associations :



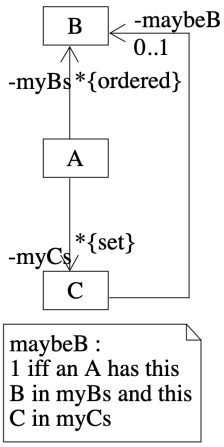
1.3 Addition : reflexive asociations

Classes can be *associated to themselves*, forming *reflexive associations*. This is an example of Russian dolls :



1.4 Addition : notes

Some things can be omitted from the class diagram (association names, navigability, visibility, types, parameter names, and even attributes and/or methods. . .). Others can be added if necessary. In an association, multiplicities can thus be decorated by a characteristic such as `ordered` or `set` (if the specific collection used to implement the association is important), and roles can also be decorated by their visibility. If an information cannot be directly given by the standard UML notations, it can always be added as a **note**. This is an example of diagram that use these decorations :



2 Exercise – Java code transcription

Here are some Java classes, transcrive this code as a class diagram indicating one-way navigability when necessary (three classes, including an association class).

```
/**
 * A simple social network
 */
public class Network {
    private Set<Member>accounts;
    /**
     * Access member accounts by name.
     * @param memberName the name of the account to search for
     * @return member account or null if absent
     */
    public Member getAccountOrNull(String memberName) {}
}
/**
 * The account of a member
 */
public class Member {
    private String name;
    private Map<Member,Relationship> friends;
    /**
     * Tests if this member has another member in their "friends" list
     * @param other the different member to check
     * @return true iff the two are connected as "friends".
     */
```

```
    public boolean knows(Member other) {}
    /**
     * Adds the other member as a friend, and reciprocate.
     * @param other the new friend
     */
    public void addToFriends(Member other) {}
}
/**
 * Class describing the relationship between two members.
 */
public class Relationship {
    private Member a, b;
    private Date initiatedOn;
    private String description="my new friend";
    public Relationship(Member a, Member b) {}
    /**
     * Creates a text representation of the relationship.
     * @return the String representation of the object
     */
    @Override
    public String toString() {}
    /**
     * Changes the description of the relationship.
     * @param desc the new description to use (String).
     */
    public void setDescription(String desc) {}
}
```

3 Lesson : the importance of code documentation

Some programmers consider that a clear code is its own documentation. However, in a project, reading the code of a method coded by another person, even very explicit and written using the norms shared by the community, remains a possible source of misunderstandings, and above all a waste of valuable time. Big projects accumulate packages, classes, and methods, and it is highly important to be able to use public methods written by others without having to analyse their code. This is the goal of in-code documentation ; different languages and environments have different norms, the Java community is based on *Javadoc*. When aiming for an useful documentation, all classes and public methods at the least must be preceded by a Javadoc header. Recall that :

- A Javadoc comment begins with `/**` and ends with `*/`.
- It contains a clear and synthetic description in one sentence (in the same human language as that used in the code, mostly English, but always in agreement with the rest of the development team).
- It contains tags `@` giving a precise semantics to some items. For a method :
  - each parameter is described by a `@param` tag followed by its name and a short description,
  - the return value (for methods that are not actions) is described by a `@return` tag,
  - if the method throws one or more exceptions, those are detailed by a `@throws` or `@exception` tag (synonyms).
- The documentation of a class should also detail :
  - the author(s) using one (or several) `@author` tag(s),
  - its version with a `@version` tag.
- Other useful tags include : `@see` (see the documentation for another class or method),`@since` (indicates the version of the project since which this documentation applies), &c. See `https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html` for a full meta-documentation.

4 Exercise – Document existing Java code

The following Java code is complete, but only one class is correctly written (with explicit code) and documented. Complete the missing documentation – these are classes for handling tenants in rooms or apartments in buildings for a real estate agency.

```
/**
 * A class representing data about a building or house.
 * @author Bruno T. Mery
 * @version 0.0
 */
public class Building {
    private String address;
    private String type;
    private Map<Pe, Pl> tenants;
    /**
     * Parameterised constructor creating an empty building.
     * @param address the post address
     * @param type a text such as "tower", "house", "dormitory"...
     */
    public Building(String address, String type) {
        this.address = address;
        this.type = type;
        tenants = new HashMap<>();
    }
    /**
     * A method that adds a new tenant in this building.
     * @param tenant the new person in the building
     * @param room the apartment/room of the new tenant
     */
    public void addTenant(Pe tenant, Pl room) {
        tenants.put(tenant, room);
        tenant.i(this, room);
    }
}

/**
 *
 *
 */
public class Pe {
    private String n;
    private Building b;
    private Pl r;
    /**
     *
     *
     */
    public Pe(String n) {
        this.n=n;
    }
    /**
     *
     *
     */
    void i(Building b, Pl r) {
        this.b=b;
        this.r=r;
        this.r.i(this);
    }
}
```

```
/**
 *
 *
 */
public class Pl {
    private int n;
    private String t;
    private Building b;
    private Set<Pe> p;
    /**
     *
     *
     *
     */
    public Pl(String t, Building b, int n) {
        this.n=n;
        this.b=b;
        this.t=t;
        this.p = new HashSet<>();
    }
    /**
     *
     *
     */
    void i(Pe p) {
        this.p.add(p);
    }
}
```